TRADING ENGINE · SLICE 2 · REFERENCE

Orders, trades, and the credit rule

Every word defined, and every case worked through with the same numbers — including the ones that go wrong. Written to be looked things up in, not read start to finish.

Before you start

- Every example uses **one order for 10 units**. Same order, same numbers, all the way through.
- The **credit** is the new idea. It does not exist in the code yet — this is the design.
- Cases are colour-coded: **green** = nothing to do, **orange** = a correction fires, **red** = something is wrong.

PART 1

01 Every word, defined

The market words

Order	Someone's offer to buy or sell. "I'll buy 10 units at £100."
Side	Whether it's a buy or a sell. Buys are **bids**, sells are **asks**.
Price	What they're willing to pay or accept, per unit.
Quantity	How much they want. Also called **amount** or **size**.
Resting order	An order sitting on the book waiting for someone to trade against it. Nothing has happened to it yet.
Order book	Every resting order, on both sides, right now. What this project reconstructs.
Price level	All the orders sitting at one particular price.
Trade	Two orders matched. A buyer and a seller agreed. Also called a **match** or an **execution**.
Fill	The same event, seen from one order's point of view. "My order got filled" = "some of my order traded away."
Partial fill	Only some of the order traded. The rest keeps resting.
Full fill	All of it traded. The order is gone.
Maker	The order that was already resting when the trade happened.
Taker	The order that arrived and immediately matched. It often never rests at all.
Snapshot	A photograph of the whole book at one instant, fetched separately from the live feed.
Seed	Building your starting book from a snapshot, before replaying anything.

The two Bitstamp channels

`live_orders`	Announces changes to resting orders: one appeared, one changed size, one went away.
`live_trades`	Announces trades: these two order ids matched, for this much.
`order_created`	A new order started resting.
`order_changed`	A resting order's remaining quantity changed. Usually because it was partly filled.
`order_deleted`	A resting order is gone. Either cancelled, or fully filled.

THE KEY FACT THIS WHOLE DOCUMENT EXISTS FOR

A single fill gets announced **twice** — once on `live_orders`, once on `live_trades`. Two messages, one real event. If you reduce the book on both of them, you've removed the same quantity twice.

The three quantity fields on a Bitstamp order message

`amount`	**How much is left right now.** Already has the fill subtracted. Shrinks over the order's life.
`amount_traded`	**How much this one event traded away.** Just this bite — not a running total.
`amount_at_create`	**How big it was originally.** Never changes.

Verified on the real capture: `previous amount − amount_traded = amount` held for 156 of 156 filled events. The running-total reading held for only 111 — exactly the ones that happened to be an order's first fill.

The code words

`OrderBook`	The real book. Holds every resting order.
`apply()`	Change the real book. Add, modify, or remove one order.
`TradeReconciler`	Keeps its own copy of what's resting, so it can work out what trades imply.
`observe()`	Tell the reconciler about an order event, so its copy stays current.
`reconcile()`	Give the reconciler a trade. It hands back the corrections the book needs.

Correction	An order event the reconciler *invents*, because the venue never sent one. Always a modify or a remove — never an add.
`Replay`	Walks both streams in timestamp order and drives everything above.
Seeded / cutoff	Where replay starts (the snapshot's time) and where it stops.

The new word

Credit	**Our own invented term — not a Bitstamp field.** A running number kept per order, meaning: "how much fill has `live_orders` told me about that `live_trades` hasn't yet?" It's how we make sure one real fill only ever reduces the book once.

PART 2

02 The rule, in full

Two things update the credit

An order event arrives carrying `amount_traded` = T

Credit goes **up** by T. (The venue already took T off the quantity it sent you.)

A trade of size Q arrives for that order

If credit ≥ Q → already handled. No correction. Credit drops by Q.

If credit < Q → correct only the uncovered part (Q – credit). Credit drops by Q, going negative.

What the credit's sign means

CREDIT	MEANING	IS THAT OK?
`positive`	`live_orders` told me about a fill; the trade hasn't arrived yet	Fine, briefly
`zero`	Both streams agree. Nothing outstanding.	The resting state
`negative`	I corrected for a trade; its order event hasn't arrived yet	Fine, briefly

THE PROPERTY WORTH REMEMBERING

Once both streams have finished describing the same fills, the credit always returns to **zero** — no matter which order the messages arrived in, or how they were grouped. A credit stuck away from zero is a signal something didn't match up.

Why this is needed at all

Without the credit, whichever message arrives second gets treated as fresh news, and the book gets reduced twice for one fill. On the real capture this deleted **5 live orders** that should still have been resting, and produced **9 errors**.

PART 3 · GROUP A

03 Both streams report the fill

The everyday case. A real fill happens, and both channels announce it. The credit's whole job is making sure the book only moves once.

Every case below: an order for 10 units is resting, credit starts at 0.

A1 Order event first, trade matches exactly

The most common case by far — and the one that's broken today.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
start	10	0	—
`order_changed` amount 4, traded 6	4	+6	—
trade of 6	4	0	**none**

Correct. The book keeps its 4. Without the credit, the reconciler sees "trade of 6 vs only 4 resting" and deletes the order entirely — losing 4 units that are really there.

A2 Trade first, order event second

Same fill, opposite arrival order.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
start	10	0	—
trade of 6	4	−6	**modify to 4**
`order_changed` amount 4, traded 6	4	0	— (no change)

Correct — and identical to A1. Same book, same credit. This is the point: the arrival order stops mattering.

A3 Trade smaller than the credit

The venue reported one fill of 6, but the trades arrive split into 5 and 1.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
`order_changed` amount 4, traded 6	4	+6	—
trade of 5	4	+1	**none**
trade of 1	4	`0`	**none**

Correct. The credit is a pot of quantity, not a list of individual fills. It doesn't care how the trades are grouped — only that they add up.

A4 Trade bigger than the credit

The mirror image: one trade of 7, but `live_orders` splits the news into 6 then 1.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
`order_changed` amount 4, traded 6	4	+6	—
trade of 7	3	−1	**modify to 3** (7 − 6 = 1 uncovered)
`order_changed` amount 3, traded 1	3	`0`	— (no change)

Correct. Only the uncovered 1 moves the book. Credit returns to zero when the matching order event lands.

A5 An order eaten in several bites

Fills of 6, then 3, then 1 — the shape actually seen in the capture.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
start	10	0	—
`order_changed` amount 4, traded 6	4	+6	—
trade of 6	4	0	none
`order_changed` amount 1, traded 3	1	+3	—
trade of 3	1	0	none
`order_deleted` amount 0, traded 1	gone	–	—
trade of 1	gone	–	none (order's gone)

Correct. Credit returns to 0 after each fill. That rhythm is the sign it's working.

A6 A delete that carries a fill

Easy to miss: `order_deleted` carries `amount_traded` too. A delete is not always a cancel — it's often the last fill.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
start (4 left after earlier fills)	4	0	—
`order_deleted` amount 0, traded 4	gone	–	—
trade of 4	gone	–	**none** — nothing to correct

Watch this one. Deletes need the same fill handling as changes. If your code only reads `amount_traded` on `order_changed`, this case silently misbehaves.

PART 3 · GROUP B

04 Only trades report the fill

These are the cases `TradeReconciler` exists for. `live_orders` stays silent, so without the trade stream the book would be permanently wrong. Here the credit is 0, so corrections fire exactly as they should.

B1 Resting order fully consumed, no delete ever sent

The original reason this component was built.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
start	10	0	—
trade of 10	gone	−10	**remove**
(`live_orders` never mentions it again)	gone	—	—

Correct. No announcement came from `live_orders`, so there was no credit, so the correction fires. The credit never gets in the way of a genuine correction.

B2 Partially consumed, no order event sent

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
start	10	0	—
trade of 6	4	−6	**modify to 4**

Correct. Note the credit sits at −6 indefinitely if the order event truly never comes. Harmless on its own — but see **E2**.

B3 Order came from the snapshot, then gets filled

The order was resting before capture started, so `live_orders` never announced it appearing. This is why `bootstrap()` takes a reconciler pointer.

WHAT ARRIVES	BOOK	CREDIT	CORRECTION
seeded from snapshot	`10`	`0`	—
trade of 10	`gone`	`−10`	**remove**

Correct — but only because seeding tells the reconciler. If `bootstrap()` is called without the reconciler, its records are empty, the trade matches nothing, and the order rests forever.

B4 Both sides of the trade are resting

A trade names two orders. Sometimes you know about both. Observed 4 times in the capture.

WHAT ARRIVES	BUY ORDER	SELL ORDER	CORRECTION
start	`10`	`10`	—
trade of 6	4	4	**two corrections**

Correct. `reconcile()` checks both the buy id and the sell id, and can return up to two corrections from one trade.

PART 3 · GROUP C

05 Cases where nothing should happen

Quiet cases. Easy to get wrong by doing something when you should do nothing.

C1 Trade names an order never seen

Someone's order arrived and matched instantly. It never rested, so it was never announced. The **taker** side of most trades.

Do nothing. No record of it, so no correction. This is expected and normal, not a missing-data problem. The book was never wrong about it — it was never in the book.

C2 Trade for an order already removed

The order left the book earlier, from a delete or a previous correction.

Do nothing. Same as C1 — no record, no correction.

C3 An order event with `amount_traded` = 0

Every `order_created` has this. So does any genuine amendment that isn't a fill.

Credit unchanged. Zero means "no fill in this message" — a real, meaningful zero. Not the same as the field being *missing*, which is an error (see **E4**).

C4 An order that's cancelled, never filled

Someone changes their mind. `order_deleted` with `amount_traded` = 0.

Remove it, credit untouched. No trade will ever arrive for this order. Nothing to reconcile.

PART 3 · GROUP D

06 Timing and ordering

Both streams carry timestamps, and they can be equal. Whoever goes first has to not break things.

D1 `order_created` and its trade share a timestamp

An order appears and is matched in the same microsecond.

IF ORDER GOES FIRST	IF TRADE GOES FIRST
The order is recorded, then the trade removes it. **Correct.**	The trade matches nothing — the order isn't recorded yet. No correction. Then the order is added and **rests forever**. **Wrong.**

This is why order events win an exact tie. The reconciler has to know about an order before a trade can ask about it.

D2 `order_changed` and its trade share a timestamp

The exact situation failing on the real capture today.

IF ORDER GOES FIRST	IF TRADE GOES FIRST
Book drops to 4. Then the trade of 6 sees only 4 resting, thinks it's a full fill, and **deletes the order. Wrong today.**	Book drops to 4 by correction. Then the order event sets it to 4 — no change. **Correct.**

The opposite of D1. No fixed tie-break can be right for both D1 and D2. That's why the fix has to be the credit, not a different ordering rule.

D3 Messages arrive far apart in time

On the real capture, gaps between a correction and the matching order event ran from 1,000 up to 36,000 microseconds.

No special handling. The credit doesn't expire or care about elapsed time. It just waits.

PART 4

07 Problems and errors

What can go wrong, how you'd notice, and what to do. The theme: **never let it be silent.**

E1 Credit stuck positive

`live_orders` reported a fill, but the trade never arrives. Credit sits at, say, +1 forever.

Why it's bad: a later genuine fill of 1 on that same order gets swallowed — "I've got 1 in credit, already heard about this." No correction fires. The book is quietly wrong.

Real evidence: two orders in the capture end with an unmatched credit of `0.06401346`. Both are the same amount — almost certainly the two sides of one trade whose message fell outside the capture window.

Do: count orders removed while still holding a non-zero credit. If the count stays around 2 per capture it's the segment boundary. If it climbs, investigate.

E2 Credit stuck negative

You corrected for a trade, but the matching order event never comes (case B1/B2 — often completely normal).

Why it's usually fine: the correction was right. Nothing is wrong with the book.

Do: nothing, but count it. A negative credit at removal is normal; a sudden change in how often it happens is worth seeing.

E3 `amount_traded` missing from a message

The field isn't there at all — malformed line, or Bitstamp changed the format.

The tempting mistake: treat it as 0. But 0 means "no fill happened," which brings the double-counting straight back — silently, with no error anywhere.

Do: return an error. A missing field is not a zero.

E4 A correction is bigger than what's resting

The uncovered part of a trade exceeds the order's remaining quantity.

Existing behaviour handles it: when a trade covers the whole remaining quantity, the reconciler emits a **remove** rather than a negative modify. Worth a test.

E5 A late delete for an already-corrected order

A correction removed the order; then `live_orders` sends its own delete.

Already handled. `Replay` recognises this and counts it as `redundantOrderRemovals` instead of failing. After the credit fix, this should become rare — if it stays common, the credit logic isn't working, and that counter is your evidence.

E6 A late *change* for an already-removed order

Same as E5, but a modify rather than a delete. **This is what crashes the run today** — all 3 aborting failures on the real capture are this.

The tempting bad fix: tolerate modifies the way E5 tolerates deletes. Then nothing crashes.

Don't. The crash isn't the problem — the wrongly deleted order is. Silencing it leaves the book wrong and stops telling you. Fix the double-counting instead.

E7 A gap, disconnect or reseed

Messages are lost. When the feed comes back, an order's quantity may have dropped for reasons you never saw.

The credit can't help here — it only knows about fills it was told about. After a gap, a quantity decrease might represent several unseen fills.

Out of scope for now, deliberately. Flagged in ADR 0013 as needing its own decision and its own evidence. Don't guess at it.

E8 Two lists drifting apart

If the loader keeps order events in one list and fill amounts in another, one skipped line puts every later fill on the wrong event.

Do: store each event and its fill **together as a pair**. Then they can't drift.

PART 5

08 What the real data shows

Measured on `data/raw/bitstamp-btcusd-20260822T000512Z` — 29,404 order events, 84 trades.

QUESTION	ANSWER
Is `amount_traded` per-event or a running total?	`per-event — 156/156`
Orders whose fill totals balance across both streams	`123 of 125`
Orders whose individual fill sizes match one-for-one	`123 of 125`
Orders that don't balance	`2 — same amount, likely one uncaptured trade`
Quantity decreases that were *not* fills	`0 of 83`
Trades where both sides were already resting	`4 of 84`
Current failures when replayed	`9 — all one cause, 5 orders`

WHAT THIS TELLS YOU

Fills almost always arrive as clean one-for-one pairs. Cases A3 and A4 — differently grouped fills — weren't observed here at all. That doesn't mean they can't happen; it means this capture can't confirm them either way, so treat those two as designed-for but unproven.

Tests worth writing, by case

TEST	PROVES
A1 — order first, exact match	The bug is actually fixed
A2 — trade first, exact match	Arrival order doesn't matter
A3 / A4 — differently grouped fills	The credit is a pot, not a matcher
A6 — delete carrying a fill	Deletes read `amount_traded` too
B1 — no delete ever sent	The credit doesn't block real corrections
B3 — snapshot order filled	Seeding reaches the reconciler
C3 — `amount_traded` = 0	Real zeros don't move the credit
D1 / D2 — shared timestamps	Both tie directions land correctly
E3 — field missing	Errors instead of silently reading 0

Any scenario, end state	**Every credit is zero when it finishes**

THE SINGLE MOST USEFUL ASSERTION

At the end of any scenario where both streams described the same fills, **every credit should be zero**. One line, and it catches most of the ways this can go wrong.